Optimización metaheurística de la configuración de extracción de métricas en fichas de modelos de Hugging Face

Juan Felipe Gallo
Maestría en Ingeniería, Universidad de Antioquia, Medellín, Colombia
Metaheurística 2026-01, Entrega 3

***Resumen*—Se optimiza la configuración de la etapa de extracción de métricas de desempeño de un *pipeline* ETL sobre fichas de modelos de Hugging Face: qué reglas de un catálogo activar, en qué orden de prioridad y con qué umbral de validación, para maximizar el F_1 macro por tarea con a lo sumo K reglas, precisión mínima de 0,90 y un tiempo máximo por ficha. Se implementa GRASP con búsqueda local de cinco movimientos y memoria de evaluaciones, y se compara con un voraz constructivo, búsqueda aleatoria y enumeración exacta. En una instancia de 12 reglas con $K = 4$, GRASP alcanza el óptimo exacto (0,316) en las cinco réplicas. En el catálogo completo de 27 reglas empata con el voraz y supera a la búsqueda aleatoria con el mismo presupuesto. En validación cruzada agrupada por organización, GRASP y el voraz no se distinguen (0,303 frente a 0,311, desviación cercana a 0,14). Una muestra manual de 36 fichas sin referencia da una precisión del 100 % (IC 95 %: 90,4–100), casi toda en formato AutoTrain. Aplicada a los 1.664 modelos sin métricas, la configuración optimizada de GRASP con $K = 6$ recupera menos de 30, mientras que usar todas las reglas recupera 1.386 (83 %): la configuración ajustada sobre la referencia no se transfiere a esa población.**

***Index Terms*—metaheurísticas, GRASP, extracción de información, fichas de modelo, optimización combinatoria.**

I. Implementación

La implementación sigue el diseño de la Entrega 2 y se organiza en los módulos de la Fig. 1. El módulo `evidence` contiene los analizadores de texto y `gold` la normalización de nombres y la comparación con la referencia; `rules` define el catálogo de 27 reglas. `evaluator` contiene `Instance`, que precalcula y guarda las salidas de las reglas y evalúa configuraciones. `baselines` aloja la clase `Objective` (aptitud, restricciones y caché de evaluaciones) y los métodos de comparación, y `grasp` implementa el algoritmo principal.

I-A. Problema y representación

Cada ficha de modelo (*model card*) [1] contiene métricas en el bloque `model-index` y, con frecuencia, en tablas y líneas del cuerpo. El ETL actual solo lee los metadatos y deja con métricas 401 de 2.057 modelos (19,5 %). Se busca configurar una etapa de recuperación sobre el cuerpo de la ficha. Una solución es $x = (\pi, \theta)$: π es una tupla ordenada de a lo sumo K reglas distintas de un catálogo de $m = 27$ y $\theta \in \{1, 100, 1000\}$ es una cota superior de plausibilidad para los valores aceptados. Con $K = 5$ el espacio tiene $3\sum_{k=1}^{5} m!/(m-k)! \approx 3{,}04 \times 10^7$ configuraciones.

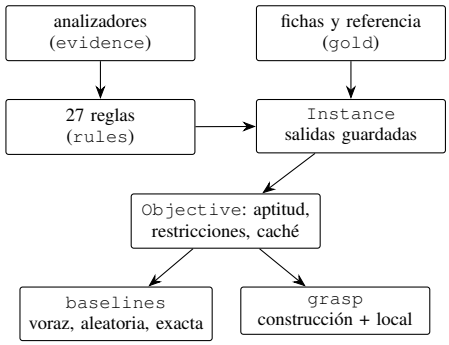


Figura 1. Componentes de la implementación y su relación.

I-B. Catálogo de reglas

Una regla combina un analizador (línea `nombre: valor`, fila de tabla *markdown* o columna de tabla, tomando la última fila) con una subfamilia de métricas: `clf_basic`, `clf_avg` (macro, micro y ponderadas), `rank` (nDCG, MAP, MRR), `at_k`, `sim`, `rouge`, `bleu`, `reg` y `lm`. Son $3 \times 9 = 27$ reglas. Medido en esta implementación, procesar una ficha con todas cuesta 187 ms, y la regla de fila de `at_k` cuesta 67,8 ms, más del doble que cualquier otra.

I-C. Evaluación con memoria

La salida de cada regla sobre cada ficha se calcula una sola vez (73 s para 27 reglas y 390 fichas), junto con las métricas de referencia que acierta. Evaluar una configuración se reduce a combinar salidas guardadas y cuesta 3,3 ms. La prioridad opera por métrica: para cada ficha y nombre normalizado de métrica, la primera regla de π que la produce es su dueña y aporta todos sus valores; las demás se descartan para esa métrica. Esto causa enmascaramiento: activar más reglas puede empeorar el resultado. Una capa adicional de memoria guarda cada configuración evaluada y el presupuesto cuenta solo evaluaciones reales.

Un valor extraído acierta si el nombre canónico coincide con el de la referencia (por ejemplo, `ROUGE-1=rouge1`, `acc=accuracy`, `ndcg at 10=ndcg@10`) y el valor coincide con tolerancia relativa del 1 % (también $\times 100$ y $\div 100$).

I-D. Función objetivo

Sea T el conjunto de etiquetas de tarea (`pipeline_tag`) y $F_{1,\tau}(x)$ el F_1 de la etiqueta τ, con precisión y cobertura agregadas en sus fichas. El objetivo es el F_1 macro, que pesa igual a cada tarea:

$$\bar{F}_1(x) = \frac{1}{|T|} \sum_{\tau \in T} F_{1,\tau}(x). \tag{1}$$

Se eligió el macro porque dos tareas aportan 226 de las 390 fichas de referencia y el F_1 agregado deja poco que optimizar (0,885 con todas las reglas sin enmascarar). Su costo es que una etiqueta con una sola ficha pesa igual que una con cientos, lo que aumenta el ruido. Las restricciones son duras: precisión global $P(x) \geq 0{,}90$ y tiempo por ficha $t(x) \leq t_{\text{máx}}$. Con la violación $v(x) = \text{máx}(0,\, 0{,}90 - P) + \text{máx}(0,\, t/t_{\text{máx}} - 1)$, la aptitud es

$$f(x) = \begin{cases} \bar{F}_1(x) & v(x) = 0, \\ -1 - v(x) & v(x) > 0, \end{cases} \tag{2}$$

de modo que toda solución infactible vale menos que cualquier factible y v orienta la búsqueda hacia la región factible. Una primera versión penalizaba el tiempo con un factor pequeño: los métodos violaban la restricción y esos resultados se descartaron.

I-E. GRASP

GRASP [2] repite construcción y búsqueda local hasta agotar el presupuesto de evaluaciones. La *construcción* evalúa en cada paso todas las inserciones (regla, posición) y elige al azar entre las que cumplen $f \geq f_{\text{máx}} - \alpha(f_{\text{máx}} - f_{\text{mín}})$; se detiene al llegar a K reglas o si ninguna inserción mejora. La *búsqueda local* aplica mejor mejora sobre cinco movimientos: insertar, eliminar, reemplazar una regla, intercambiar dos posiciones y cambiar θ. Devuelve la mejor configuración.

I-F. Líneas base

Voraz individual: las K reglas con mejor aptitud individual, en ese orden (línea base propuesta en la Entrega 1). *Voraz constructivo*: determinista, inserta en cada paso la regla y posición que más mejoran. *Búsqueda aleatoria*: configuraciones aleatorias con el mismo presupuesto. *Enumeración exacta*: todas las ordenaciones de hasta K reglas de un subcatálogo, para cada θ.

II. Experimentos

II-A. Instancias

Se descargaron 1.931 de 2.057 fichas; 126 no fueron accesibles (94 con error 403 y 32 con 404). La referencia es el `model-index` de 390 fichas con valores numéricos, oculto al extractor (se elimina el bloque YAML inicial). La partición se hizo por organización. El supuesto de representatividad de la Entrega 1 resultó falso: las fichas con métricas en el ETL son sobre todo de similitud de oraciones y extracción de características, y las 1.533 sin ellas son en su mayoría AutoTrain. Por eso se añadió una muestra manual de 36 fichas sin referencia, proporcional por etiqueta. Las instancias son:

Cuadro I
Aptitud media de GRASP ($K = 5$, 6 semillas, entrenamiento)

α	500	1500	3000
0,0	0,386	0,398	0,398
0,1	0,355	0,418	0,418
0,3	0,347	0,416	**0,420**
0,5	0,347	0,416	0,420
0,8	0,343	0,418	0,418

- **Pequeña con óptimo exacto:** subcatálogo de 12 reglas, $K = 4$.
- **Catálogo completo:** 27 reglas, $K \in \{3, 4, 5, 6\}$, con $t_{\text{máx}} \in \{30, 50, 80\}$ ms o sin límite.
- **Validación cruzada por organización:** 5 pliegues balanceados por número de fichas (236, 39, 39, 38 y 38). Una sola organización aporta 236 de las 390 fichas y forma el primer pliegue completo.

II-B. Ajuste de parámetros

Se ajustaron $\alpha \in \{0; 0{,}1; 0{,}3; 0{,}5; 0{,}8\}$ y el presupuesto $\in \{500; 1500; 3000\}$ evaluaciones, con 6 semillas por combinación, sobre 338 fichas de entrenamiento ($K = 5$, $t_{\text{máx}} = 80$ ms). El criterio fue la aptitud media (Tabla I). Se eligió $\alpha = 0{,}3$ y 3.000 evaluaciones (0,420). Con 500 evaluaciones no se completa una iteración (la primera cuesta unas 700). De 1.500 en adelante hay meseta, y para $\alpha \geq 0{,}1$ la diferencia es menor que 0,004; $\alpha = 0$ (voraz más búsqueda local) queda ligeramente por debajo.

II-C. Protocolo

Presupuesto de 3.000 evaluaciones (1.000 en la instancia pequeña). Réplicas: 5 semillas en la instancia pequeña y con restricción de tiempo, 10 en la curva de convergencia y 3 por pliegue en validación cruzada. Se reportan $\bar{F}_1$, precisión, cobertura, tiempo y factibilidad. El hardware y los tiempos son los de esta implementación en Python.

II-D. Desviaciones respecto a la Entrega 1

El catálogo tiene 27 reglas y no entre 40 y 60. Se probaron $K \in \{3, 4, 5, 6\}$ y no $\{5, 10, 15\}$, y solo la precisión mínima de 0,90 (no 0,85 y 0,95). La referencia son 390 fichas con valores numéricos y no 401, y la tabla de homologación se sustituyó por una normalización canónica de nombres. No se escalaron subconjuntos de 50, 100 y 200 fichas ni subcatálogos de 20 y 40 reglas; en su lugar hay una instancia pequeña con óptimo exacto y una validación cruzada por organización. La línea base ya no es la cascada actual, que solo lee metadatos, sino los métodos voraces y la búsqueda aleatoria sobre las mismas reglas. El supuesto de representatividad no se sostuvo, como se indicó arriba.

III. Resultados

III-A. Óptimo exacto

Con 27 reglas y $K = 3$, la enumeración completa (54.837 configuraciones, 43 s) da 0,264, que el voraz constructivo ya

Cuadro II
$\bar{F}_1$ POR MÉTODO Y K (CATÁLOGO COMPLETO)

K	Voraz ind.	Voraz constr.	Aleatoria	GRASP
3	0,199	0,264	–	–
4	0,199	0,316	–	–
5	0,242	0,340	0,286	0,340
6	–	0,357	0,320	0,357

Cuadro III
$\bar{F}_1$ Y TIEMPO MEDIO POR FICHA CON $K = 5$ (5 SEMILLAS)

	Voraz constr.		Aleatoria		GRASP	
$t_{\text{máx}}$	$\bar{F}_1$	ms	$\bar{F}_1$	ms	$\bar{F}_1$	ms
30	0,246	29	0,213	23	0,247	28
50	0,288	47	0,242	44	0,288	47
80	0,285	78	0,260	69	0,289	54

alcanza. En el subcatálogo de 12 reglas con $K = 4$, el óptimo exacto (75 s) es 0,316; el voraz y GRASP lo alcanzan en las 5 réplicas, y la búsqueda aleatoria con 1.000 evaluaciones logra 0,294 en promedio (0,274–0,306). Con $t_{\text{máx}} = 50$ ms, el óptimo en esa instancia es 0,264.

III-B. Catálogo completo

La Tabla II compara métodos sin límite de tiempo. El voraz individual se estanca porque elige reglas que se solapan (0,199 con $K = 3$ y $K = 4$). El voraz constructivo y GRASP coinciden, y la búsqueda aleatoria queda por debajo. La cota superior (todas las reglas sin enmascarar) es 0,383.

III-C. Restricción de tiempo

Sin límite de tiempo, el voraz usa la regla de 67,8 ms y llega a 0,340 con 81 ms por ficha. La Tabla III muestra el compromiso con $K = 5$; todas las soluciones son factibles. GRASP iguala al voraz con 30 y 50 ms, y con 80 ms es ligeramente mejor (0,289 frente a 0,285) usando 54 ms en lugar de 78 ms. A 80 ms el voraz también cae por debajo de lo que logra a 50 ms (0,285 frente a 0,288): no cabe la regla cara y cae en una combinación peor.

III-D. Convergencia

La Figura 2 muestra la mejor aptitud acumulada ($K = 5$, $t_{\text{máx}} = 80$ ms, 10 semillas). GRASP produce su primera solución cerca de las 700 evaluaciones, alcanza 0,283 a las 800 y se estabiliza en 0,288 desde las 1.600, con una banda muy estrecha. La búsqueda aleatoria llega a 0,264 a las 3.000 evaluaciones. GRASP converge evaluando unas 1.000 configuraciones, cerca del 0,003 % del espacio.

III-E. Validación cruzada por organización

En cada pliegue se optimiza con los otros cuatro y se evalúa en el reservado ($K = 5$, $t_{\text{máx}} = 80$ ms, $\alpha = 0{,}3$, 3.000 evaluaciones). La Tabla IV resume los pliegues. Los valores varían mucho entre pliegues: el techo de cada uno (todas las reglas sin enmascarar) va de 0,22 a 0,74. Las diferencias pareadas entre GRASP y el voraz son $-0{,}033$, $-0{,}159$, $+0{,}055$, $-0{,}044$ y $+0{,}138$ (media $-0{,}008$): no hay evidencia de que uno supere al otro. Entre GRASP y la aleatoria la media es $+0{,}032$ y gana en 3 de 5 pliegues. El voraz supera a la aleatoria en 4 de 5 (media $+0{,}040$).

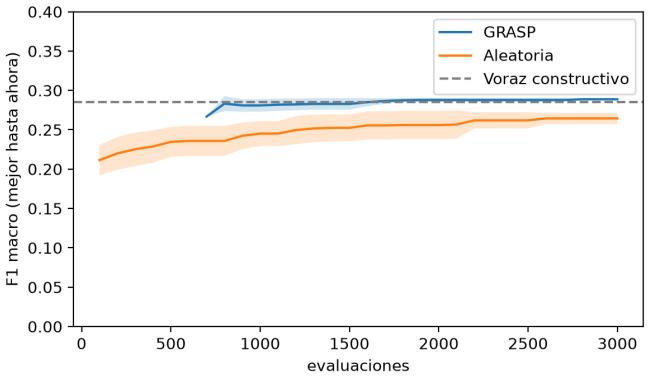


Figura 2. Mejor $\bar{F}_1$ acumulado frente a evaluaciones reales ($K = 5$, $t_{\text{máx}} = 80$ ms; media $\pm$ desviación de 10 semillas).

Cuadro IV
VALIDACIÓN CRUZADA (MEDIA $\pm$ DESV. ENTRE 5 PLIEGUES)

Método	$\bar{F}_1$	Precisión	Cobertura
Voraz constructivo	$0{,}311 \pm 0{,}137$	0,991	0,485
GRASP	$0{,}303 \pm 0{,}144$	0,954	0,312
Aleatoria	$0{,}271 \pm 0{,}109$	0,971	0,338

III-F. Muestra manual sin referencia

De 36 fichas, 34 siguen la plantilla de AutoTrain y 2 son de formato libre. Se comparó cada valor extraído con el texto de la ficha: las 36 son correctas. El intervalo de Wilson [3] al 95 % es 90,4–100 %. En 4 fichas el extractor omitió métricas (RMSLE, varianza explicada, CodeBERT y columnas de otros conjuntos de datos). En total el extractor encuentra alguna métrica en 1.725 de 2.057 fichas (84 %), 1.387 de ellas sin referencia, frente al 19,5 % de la cascada actual.

III-G. Aplicación a los modelos sin métricas

Se aplicaron dos configuraciones a los 1.664 modelos sin valores numéricos en el `model-index` (126 sin README). La de GRASP ($K = 6$, 80 ms; $\bar{F}_1 = 0{,}340$, $P = 0{,}995$ y cobertura 0,707 en la referencia) recupera métricas en menos de 30 de ellos. Con todas las reglas en el orden fila, columna, línea ($\bar{F}_1 = 0{,}384$, 170 ms por ficha) se recuperan en 1.386 (83 %): 88 % de las fichas de clasificación de texto, 94 % de imágenes, 90 % de resumen, 95 % de clasificación de tokens y 91 % de traducción; en generación de texto solo 9 %. Una inspección de 12 fichas al azar entre las 1.386 no mostró valores ajenos a las métricas del modelo.

IV. ANÁLISIS Y CONCLUSIONES

Enmascaramiento. En una versión previa de 18 reglas, activarlas todas en el orden del catálogo daba cobertura 0,374, frente a 0,801 con un orden adecuado (fila, columna, línea); sin

enmascarar también 0,801. El orden de prioridad es entonces una decisión real y no basta con activar todo.

Métodos. Un voraz constructivo es una línea base muy fuerte aquí. GRASP lo iguala y, en la instancia pequeña, alcanza el óptimo exacto en todas las réplicas. Su ventaja más clara es con la restricción de tiempo a 80 ms, aunque de solo 0,004. Ambos superan a la búsqueda aleatoria en el conjunto completo. Fuera de muestra, la diferencia GRASP–voraz es ruido y GRASP tiene menor cobertura (0,312 frente a 0,485). Una posible explicación, no comprobada, es sobreajuste: el F_1 macro pesa por igual etiquetas con una o dos fichas, y una búsqueda más intensa se ajusta a ese ruido.

Transferencia. La configuración optimizada no se transfiere. Se ajustó sobre una referencia de similitud de oraciones y extracción de características, y con $K = 6$ queda sin las reglas en línea que usa el formato AutoTrain, que domina la población objetivo. Un límite de K pensado para el mantenimiento sacrifica así casi toda la cobertura donde más hace falta. La función objetivo debe incluir esa población, por ejemplo con los conjuntos validados de AutoTrain o con una ponderación por la distribución real de tareas.

Limitaciones. (i) La precisión es una cota inferior, porque el cuerpo puede traer métricas correctas ausentes del `model-index`. (ii) En las fichas AutoTrain `model-index` y cuerpo reportan mediciones distintas (por ejemplo, ROUGE-2 de 12,47 frente a 10,90), lo que debilita la independencia de la referencia. (iii) Una sola organización aporta 236 de las 390 fichas y cerca del 75 % de las métricas de referencia de la partición de entrenamiento usada en el ajuste. (iv) Los pliegues son muy desiguales y varias etiquetas tienen una sola ficha. (v) Los hiperparámetros se ajustaron en una partición distinta de la validación cruzada, con posible fuga leve; el efecto es pequeño porque α no es sensible. (vi) Los tiempos dependen de la implementación y la máquina. (vii) La muestra manual es casi toda AutoTrain y no estima la precisión en formato libre.

Trabajo siguiente. La Entrega 4 incorpora un algoritmo genético y una prueba estadística pareada sobre más instancias, un catálogo mayor que amplíe el techo de 0,383, una ponderación por ficha para reducir el peso de las organizaciones dominantes y de las etiquetas diminutas, y una función objetivo que incluya los modelos AutoTrain.

REFERENCIAS

[1] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT*)*, Atlanta, GA, USA, 2019, pp. 220–229.

[2] T. A. Feo and M. G. C. Resende, "Greedy randomized adaptive search procedures," *J. Global Optim.*, vol. 6, no. 2, pp. 109–133, 1995.

[3] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.